Model Inversion Attack Report

1 Objective

The goal of this experiment was to perform a **Model Inversion Attack** on a trained convolutional neural network (CNN) model, in order to reconstruct representative input images for specific target classes from the model's learned parameters.

The experiment was implemented using two scripts:

- `inversion.py` — for generating class-representative reconstructions via optimization.
- `check_prediction.py` — for verifying the model's confidence and logits on the reconstructed images.

The inversion process aims to demonstrate how an adversary, with access to a model's weights, can extract visual representations of the data the model was trained on, potentially leaking private or sensitive information.

—

2 Background and Concept

A **Model Inversion Attack (MIA)** reconstructs input data or its visual characteristics from a model's outputs or gradients.

When a neural network is trained, it learns parameters that encode the distribution of its training data. If an attacker gains access to the model and its gradients or confidence scores, they can optimize a random input image to produce a specific model output — effectively "inverting" the model to recover the input representation.

Mathematically, the objective is:

$$x^* = \arg\min_x \mathcal{L}(x) = -f_\theta(x)[t] + \lambda_{L2}\|x\|_2^2 + \lambda_{TV}\mathrm{TV}(x)$$

where:

- $f_\theta(x)[t]$ is the model's logit for target class t,
- $\mathrm{TV}(x)$ is the total variation loss (smoothness prior),
- λ_{L2} and λ_{TV} are regularization weights.

The optimization seeks to maximize the model's confidence for a specific target class while keeping the image smooth and natural-looking.

—

3 Methodology

3.1 Model and Setup

- **Model:** SimpleCNN trained on CIFAR-10.
- **Checkpoint:** `best_cnn_cifar10.pth`.
- **Image Size:** 32×32×3 (RGB).
- **Optimization Steps:** 2000 iterations per restart.
- **Learning Rate:** 0.1 (Adam optimizer).
- **Regularization Weights:** $\lambda_{L2} = 10^{-3}$, $\lambda_{TV} = 10^{-3}$.
- **Restarts:** 3 independent runs to find the best image.

—

3.2 Inversion Process

The inversion was implemented using a gradient-based optimization loop in the `invert_target()` function.

Step 1: Initialization

Each run begins with a random noise image:

$$x_0 \sim \mathcal{U}(0, 1)$$

This random image is optimized to represent the target class by maximizing the corresponding output logit.

Step 2: Forward and Loss Computation

The image is normalized and passed through the model:

$$z = f_\theta(\text{normalize}(x))$$

The loss to minimize is:

$$\mathcal{L}(x) = -z_t + \lambda_{L2}\|x\|_2^2 + \lambda_{TV}\text{TV}(x)$$

Regularization terms:

- **L2 loss:** Prevents pixel values from growing excessively.
- **Total Variation (TV) loss:** Enforces spatial smoothness and removes noise:

$$\text{TV}(x) = \frac{1}{HW}\sum_{i,j} |x_{i+1,j} - x_{i,j}| + |x_{i,j+1} - x_{i,j}|$$

Step 3: Optimization and Clamping

The optimizer updates the image pixels using gradient descent:

$$x_{k+1} = \text{clamp}(x_k - \eta \nabla_x \mathcal{L}(x_k))$$

where clamping keeps pixel values within [0,1].

Step 4: Multiple Restarts and Best Selection

To avoid local minima, three random restarts are performed. After each run, the image with the highest logit for the target class is saved as:

`inv_target<t>_best.png`

Example Code Snippet:

```
for r in range(restarts):
    img = torch.rand(1, C, H, W, device=device).requires_grad_(
       True)
    optim_img = optim.Adam([img], lr=lr)
    for it in range(steps):
        logits = model(normalize_batch(img, mean, std))
        target_logit = logits[0, target]
        loss = -target_logit + l2 * (img**2).mean() + tv *
           tv_loss(img)
        optim_img.zero_grad()
        loss.backward()
        optim_img.step()
        img.data.clamp_(0, 1)
```

—

4 Results

The inversion and verification outputs are shown below.

4.1 Inversion Output

From running:

```
python3 inversion.py --ckpt ./results/best_cnn_cifar10.pth
```

The console output was:

```
Using device: cpu
Restart 1: best logit for class 0: 44.4405 (eval 44.4399)
Restart 2: best logit for class 0: 42.5030 (eval 42.5023)
Restart 3: best logit for class 0: 45.3519 (eval 45.3523)
Saved best reconstruction for class 0 (logit 45.3523)
-> ./inversions/inv_target0_best.png
```

The model achieved a very high logit (45.35) for class 0, indicating that the reconstructed image strongly activates that class neuron.

—

4.2 Prediction Verification

The reconstructed image was verified using:

```
python3 inversion_test.py \
  --ckpt ./results/best_cnn_cifar10.pth \
  --img ./inversions/inv_target0_best.png
```

The top-5 predictions were:

```
Top-5 predictions (class : prob)
0 : 1.000000 (logit 45.3347)
7 : 0.000000 (logit -14.1354)
8 : 0.000000 (logit -16.3228)
4 : 0.000000 (logit -16.8939)
1 : 0.000000 (logit -19.8734)

All logits:
['45.3347', '-19.8734', '-20.7635', '-26.7472',
 '-16.8939', '-28.0401', '-23.6672', '-14.1354',
 '-16.3228', '-20.9585']
```

4.3 Interpretation of Results

- The reconstructed image (`inv_target0_best.png`) was classified as class 0 with **100% probability** and logit value of approximately **45.33**.
- The logits for other classes are highly negative (from -14 to -28), showing that the generated image is extremely biased toward the target class.
- The success of the inversion demonstrates that the model retains significant information about the visual structure of the target class in its parameters.
- The reconstructed image visually represents the core features that the CNN associates with class 0 (likely an airplane or automobile depending on the CIFAR-10 label mapping).

—

5 Discussion

5.1 Attack Effectiveness

The inversion attack successfully recovered an image that the model recognizes with almost absolute confidence (probability = 1.0). This suggests that:

- The model's internal representations contain enough information to reconstruct class-level prototypes.
- The CNN's parameters implicitly store approximate visual templates for each class.

5.2 Factors Affecting Reconstruction Quality

- **Regularization:** Both L2 and TV regularization terms were essential for obtaining smooth and meaningful images rather than pure noise.
- **Learning Rate:** A moderate value (0.1) helped stable convergence.
- **Restarts:** Multiple random restarts prevented local minima and improved the best-found solution.

5.3 Privacy Implications

The results reveal a potential privacy risk: a malicious actor with access to the trained model could reverse-engineer approximate visual representations of its training data. While the attack reconstructs generalized prototypes rather than exact training samples, in other contexts (e.g., facial recognition), such inversions can leak sensitive personal data.

5.4 Trade-off Between Attack Success Rate and Regularization Metric

A key trade-off observed in model inversion attacks lies between **attack success rate** and the **selected regularization metric**

As regularization strength increases (i.e., higher λ_{L2} or λ_{TV} values), the reconstructed images become smoother and more natural-looking due to stronger suppression of high-frequency noise. However, this smoothness comes at the cost of **reduced attack success rate**, since excessive regularization constrains the optimization from fully exploiting the model's gradients, resulting in lower logits and weaker class activations.

Conversely, when the regularization coefficients are reduced, the optimization more aggressively maximizes the target logit, leading to a higher attack success rate — reflected in greater confidence and larger logit values for the target class. The downside is that such reconstructions often contain noticeable artifacts or unrealistic patterns, as the model prioritizes activation maximization over perceptual quality.

Therefore, achieving effective inversion requires balancing fidelity and naturalness:

- Lower regularization $\rightarrow$ higher attack success but lower image realism.
- Higher regularization $\rightarrow$ more realistic reconstructions but reduced attack success.

This trade-off underscores the importance of tuning the regularization weights to achieve a compromise between interpretability and the degree of privacy leakage.

—

6 Conclusion

The Model Inversion Attack demonstrated that:

- The trained CNN can be inverted to produce class-representative images.
- The reconstructed image for class 0 achieved a confidence of 1.0 and a logit of 45.35, showing strong alignment with the target class.
- The attack highlights how deep models can leak embedded knowledge about training data distributions.

Although this attack does not recover exact training examples, it reveals approximate visual features of classes, underlining the importance of privacy-preserving training techniques such as differential privacy or model sanitization.